Crypto Order Book Engine

System Design & Architecture Documentation

A production-grade, real-time order book built in Rust. WebSocket ingestion from Binance, async Tokio throughout, REST API with Axum. Every decision documented. Every tradeoff named.

Language: Rust | Runtime: Tokio | Exchange: Binance | API: Axum

Part 1 -What Are We Actually Building

Binance is a crypto exchange. When people buy and sell Bitcoin, those orders sit in a list called an **order book** -bids on one side (people wanting to buy), asks on the other (people wanting to sell). That list changes thousands of times per second.

We are building a program that does three things:

1. Connects to Binance and receives those changes in real time.

2. Keeps its own copy of the order book in memory, always up to date.

3. Lets other programs ask what the order book looks like right now, over HTTP.

Simple goal. Hard to do correctly at production quality. This document explains every decision we made and exactly why we made it.

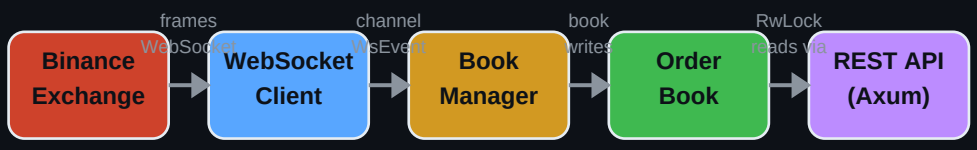

Figure 1 -Top-level data flow

Part 2 -Cargo.toml: The Dependency Decisions

Every dependency is a bet -you're betting this tool is mature, maintained, and won't fight with the rest of your system. Here is every bet we made and why.

tokio

Rust by default does one thing at a time. If you are waiting for Binance to send you a message, your program just sits there doing nothing. Tokio is the system that lets your program do many things at the

same time: listen to Binance, handle an incoming HTTP request, run a timer, all without needing multiple CPU cores. It is the foundation everything else sits on. Without it, none of the other choices make sense.

tokio-tungstenite

Binance sends us data over a WebSocket. A WebSocket is like a phone call: you open the connection once and both sides can talk back and forth as long as they want, instead of making a new request every time like normal web traffic. tokio-tungstenite speaks that protocol. We picked this one specifically because it is built to work with Tokio. They use the same underlying machinery so they do not fight each other.

futures-util

When Binance sends us a message over the WebSocket connection, it does not arrive all at once with a neat label on it. It arrives as a stream, like water from a tap rather than a package being delivered. futures-util gives us the tools to work with that stream: read the next item, process it, wait for the next one.

serde and serde_json

Binance sends us JSON. It looks like this:

```
{"e":"depthUpdate","E":1699000000000,"s":"BTCUSDT","U":123456,"u":123460,...}
```

We need to turn that text into actual Rust values we can work with. serde is the system that does that conversion automatically. You describe the shape of what you expect, it reads the JSON and fills in the values. This is the Rust standard. Every serious Rust project uses it.

rust_decimal

This one is non-negotiable. Binance sends prices as strings like "29431.50000000". The obvious thing to do is convert that to a floating point number. But floating point numbers cannot represent most decimal values exactly. 0.1 + 0.2 in floating point does not equal 0.3, it equals 0.30000000000000004. In a calculator app that is annoying. In a financial system that is a lawsuit. rust_decimal stores the number exactly as written. We never use floats for prices, ever.

thiserror

When something goes wrong, different things going wrong need different responses. The network dropped? Reconnect. Got a message we could not parse? Log it and keep going. The order book got out of sequence? Throw everything away and start over from scratch. thiserror lets us define a list of named failure types so that when something breaks, the code that catches it can look at which type it is and react correctly. Without it, we would have one generic error that tells us nothing about what to do.

axum

This is the HTTP server. When another program wants to ask what the current order book looks like, they send an HTTP request and we send back the answer. Axum handles that. We picked Axum because it is built by the same team as Tokio and works with it natively, instead of other options that have their own competing runtime underneath.

tracing and tracing-subscriber

In a normal program you would just print to see what is happening. But our program does many things simultaneously. A regular print statement has no way to tell you which of those things the message came from. tracing is logging that understands concurrency. Every message knows what task it came from, what connection it was on, what it was in the middle of doing. When something goes wrong at 3am, this is how you find out why.

url

Instead of building the Binance WebSocket address by gluing strings together, we use a typed URL that validates the address as we build it. String concatenation for URLs is how you get subtle bugs like a missing slash or a wrong character that only show up at runtime. This catches them at startup.

Tradeoff Summary

Decision	We Chose	Alternative	Why This One
Price type	rust_decimal	f64 float	Floats have rounding errors. Financial systems cannot have silent rou
HTTP framework	axum 0.7	actix-web	actix-web has its own actor runtime that conflicts with tokio. axum is t
Error handling	thiserror	anyhow	anyhow erases error types. We need to match on error variants to kn
Logging	tracing	println / log	println has no concept of which async task it came from. tracing track

Part 3 -error.rs: The Failure Taxonomy

We wrote the error types before any real logic. This is deliberate. If you define what can go wrong before you write the code, every piece of code knows exactly what failures it is allowed to produce. If you do it the other way around, you end up with scattered, incoherent errors that have no relationship to each other.

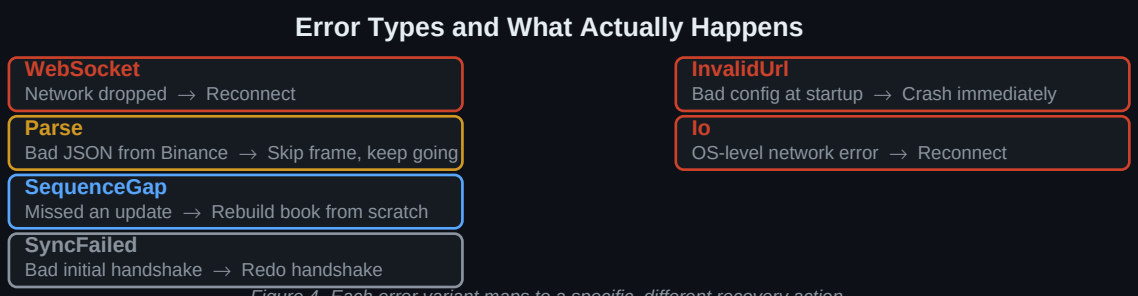

Figure 4 -Each error variant maps to a specific, different recovery action.

WebSocket

The phone call with Binance dropped. Maybe their server restarted, maybe your internet hiccupped, maybe a router between you died. When this happens: hang up and call back. The order book is still valid -we just need a new connection.

Parse

Binance sent us a message we could not understand. Maybe our code has a bug in how we described the expected shape. Maybe Binance sent something unusual. When this happens: log the bad message, throw it away, keep the connection open, keep going. One bad frame does not invalidate the entire stream.

SequenceGap

Binance does not send us the full order book every time -that would be too much data. Instead they send changes: remove this price level, add this one, update that one. Each change has a sequence number so we can tell if we missed any. If we get number 100 then number 102 without ever seeing 101, we have lost a change and our book is now wrong -we do not know what 101 was. When this happens: throw away

everything and rebuild the book from scratch.

SyncFailed

When we first connect, there is a specific handshake we do with Binance to make sure our copy of the book and their actual book are starting from the same point. If that handshake fails -if the numbers do not add up -we cannot trust anything. When this happens: start the handshake over.

InvalidUrl

The address we are trying to connect to is malformed. This only ever happens at startup if the configuration is wrong. When this happens: crash immediately with a clear message. There is no recovering from bad configuration -you need a human to fix it.

Io

Low-level network errors from the operating system itself. Connection refused, address unreachable, that kind of thing. When this happens: treat it the same as a dropped WebSocket and reconnect.

Part 4 -ws/mod.rs: The Channel Contract

The WebSocket client and the order book manager are two separate parts of the system that run independently and never call each other directly. They communicate through a queue -the client puts things in, the manager takes things out. WsEvent defines everything that can ever go in that queue. It is a contract between two parts of the system that do not otherwise know about each other.

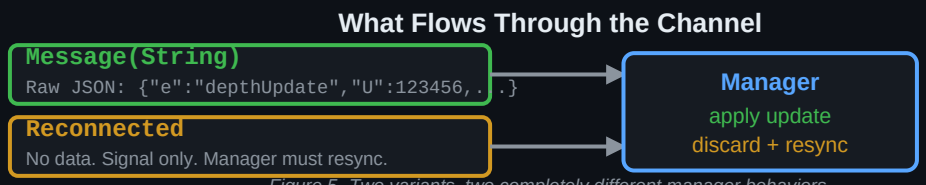

Figure 5 -Two variants, two completely different manager behaviors.

Message(String)

Binance sent us something. Here is the raw text, exactly as received. We do not parse it here -that is the manager's job. The WebSocket client's only responsibility is to get the bytes from Binance to the queue. Keeping parsing out of the client means if Binance ever changes their JSON format, we fix it in one place and the client never needs to change.

Reconnected

We lost the connection and re-established it. Whatever the manager knew before is now potentially stale -the book might be based on a stream that is no longer continuous. Start over. This signal arrives in the queue before any new messages, which is critical -if new messages arrived first, the manager would try to apply them to a book that might be from a completely different session.

Part 5 -ws/client.rs: The Reconnecting WebSocket Client

This is the piece that actually talks to Binance. It runs forever in the background, maintaining the connection and feeding messages into the queue. Here is every decision behind it.

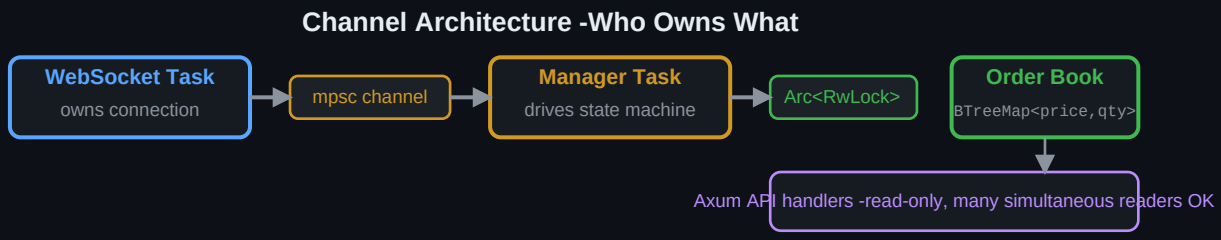

Figure 3 -One writer (manager), many readers (API). The channel enforces ordering.

Why it runs in its own independent task

When we call spawn, we hand the connection off to a background process and walk away. We do not sit there waiting for it to finish -it never finishes, that is the point. It runs forever in the background, feeding us messages. If we did not do it this way, the act of listening to Binance would block everything else in our program from running.

The reconnect loop -what actually happens

The outer loop is simple: try to connect. If it works, read messages until it breaks. When it breaks, try to connect again. The complexity is in how long we wait between attempts.

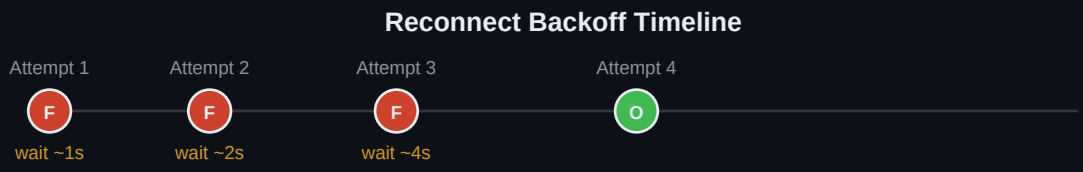

Figure 2 -Each failure doubles the wait. Jitter desynchronizes simultaneous reconnects.

Why we wait at all -and why the wait grows

Imagine Binance's servers go down. If every client immediately tries to reconnect and keeps trying every millisecond, Binance's servers come back up and immediately get flooded by millions of simultaneous reconnect attempts -which might knock them down again. We wait. And we wait longer each time we fail, because if we have failed three times in a row, something is genuinely wrong and hammering the server harder is not going to fix it.

Why we add randomness to the wait

Even with growing wait times, if ten thousand clients all started at the same moment they will all be on the same schedule. They all wait 1 second, all try at once, all fail, all wait 2 seconds, all try at once again. Adding a small random amount to each client's wait time spreads them out so they do not all hit at the exact same millisecond.

Why we cap the wait at 30 seconds

Pure doubling would eventually mean waiting hours between attempts. That is too long -if Binance comes back up after 5 minutes we want to reconnect within 30 seconds, not be stuck waiting another hour. 30 seconds is the ceiling.

The safety math -what happens after 64 failures

The wait time formula doubles each time. After 64 failed attempts, 2 to the power of 64 is a number so large it does not fit in the variable storing it. In most languages that silently wraps around to zero -no wait at all -exactly the wrong behavior when we are trying to avoid hammering a broken server. We used arithmetic that, when it would overflow, just stays at the maximum value. Then we cap it at 30 seconds. Two layers of protection.

When we send Reconnected -and why it must be first

When we successfully reconnect after a failure, the very first thing we put in the queue is Reconnected. Before any messages. This tells the manager: everything you knew is potentially wrong, start over. If we sent messages first, the manager would try to apply them to a stale book from the previous session. The ordering of events in the queue is the guarantee.

When we shut down cleanly

We check every time we try to put something in the queue whether the other side is still listening. If the manager has gone away and nobody is reading from the queue, we stop. We do not keep running a background task that produces output nobody will ever consume.

Decision	We Chose	Alternative	Why This One
Task model	spawn (fire and forget)	return JoinHandle	Returning a handle implies the caller manages the lifecycle. This task
Reconnect signal	WsEvent::Reconnected in	Separate control channel	Two channels = possible race where a stale message arrives after th
Jitter source	System clock nanoseconds	rand crate	We only need desynchronization, not cryptographic randomness. Pul
Overflow safety	saturating_mul / saturating_pow	unchecked	After 64 failures unchecked math wraps to zero -producing instant ret

Part 6 - ws/binance.rs: Parsing What Binance Actually Sends

Every few milliseconds Binance sends our program a message describing what just changed in the order book. This file is the one place in the entire system responsible for reading that message and turning it into something the rest of our code can understand. Nothing else in the system ever sees Binance's raw format.

What a Live Order Book Looks Like

PRICE	BIDS (buyers) QUANTITY	PRICE	ASKS (sellers) QUANTITY
29,501.00	0.842	29,502.00	0.310
29,500.00	1.245	29,503.00	2.100
29,499.50	3.100	29,504.50	0.750
29,498.00	0.500	29,507.00	5.000
29,495.00	12.300	29,510.00	8.200

SPREAD = $1.00

Figure 6 - The spread is the gap between the highest buyer and lowest seller. Trades happen when they meet.

What an order book actually is

An order book is two sorted lists sitting next to each other. On the left side are all the people who want to buy Bitcoin and what price they are willing to pay. The highest buyer is at the top. On the right side are all the people who want to sell Bitcoin and what price they want. The lowest seller is at the top. The gap between the highest buyer and the lowest seller is called the spread. When a buyer and seller agree on a price, a trade happens and both entries disappear.

What Binance actually sends us

Binance does not send us the whole order book every time something changes. That would be a massive amount of data, thousands of entries, sent thousands of times per second. Instead they send a small update message describing only what changed since the last message. It might say: the quantity of buyers at price 29500 is now 1.245 Bitcoin, and the entry at price 29499.50 no longer exists.

Figure 7 - Each field in the JSON has a specific purpose. We only use the ones relevant to book maintenance.

How Binance signals a deletion

When a price level needs to be removed from the book, Binance does not send a separate delete message. They send the same update format but with a quantity of zero. That zero is the signal. Our code checks for it and removes that price from our copy of the book instead of updating it. If we missed this and stored price levels with zero quantity, our book would fill up with ghost entries that do not actually exist on the exchange.

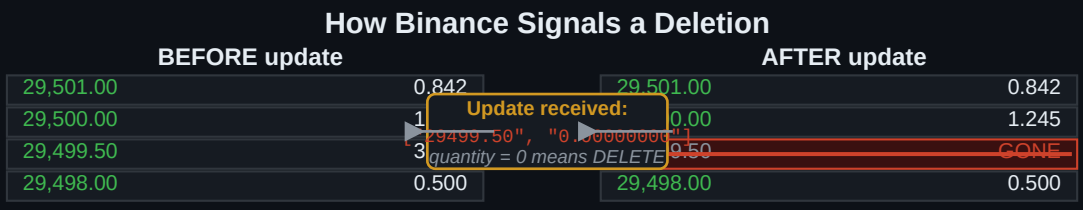

Figure 8 - Quantity zero is Binance's delete signal. Missing this would leave ghost entries in our book.

Why prices arrive as text strings, not numbers

Binance sends every price and quantity as a piece of text: "29500.00000000" rather than the number 29500. This is intentional on their part. The moment you turn that text into a standard computer number, you risk tiny rounding errors. Computers store most decimal numbers approximately, not exactly, because of how binary math works. We parse the text directly into a decimal type that preserves the exact value. This is not optional in financial software.

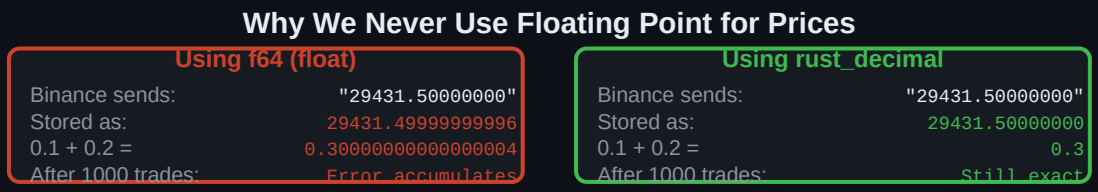

Figure 9 - Floating point errors are invisible and accumulate. In financial software, exact is not optional.

The two-layer design: wire format vs domain type

We define two completely separate sets of types in this file. The first set, RawDepthUpdate and the string arrays, match exactly what Binance sends. They are only used inside this file and are never seen by the rest of the system. The second set, DepthUpdate and PriceLevel, are what our system actually works with. They have proper numeric types, clear field names, and no trace of Binance's naming. The conversion happens in one function called parse_depth_update. If Binance changes their JSON format tomorrow, we change this file and nothing else.

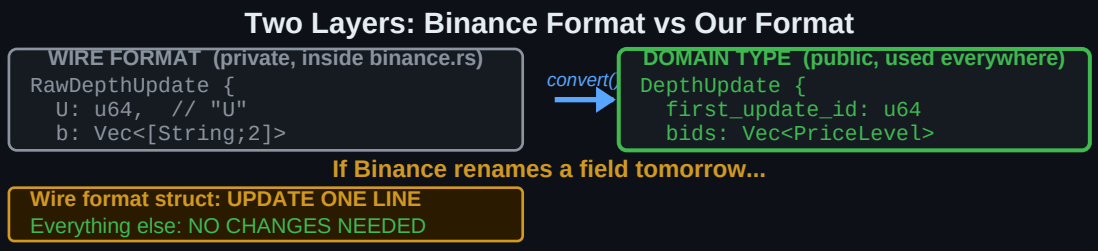

Figure 10 - The wire format and domain type are kept separate on purpose. One adapts to Binance; the other belongs to us.

The sequence numbers and why they matter

Every update message from Binance contains two numbers: the ID of the first change bundled in this message, and the ID of the last change. These are like page numbers in a book. If you receive page 5 and then page 7, you know page 6 is missing. You cannot reconstruct the story with a missing page, so you have to start the book over. Our manager will use these numbers to detect any gaps. This file's job is just to carry those numbers faithfully into our domain type.

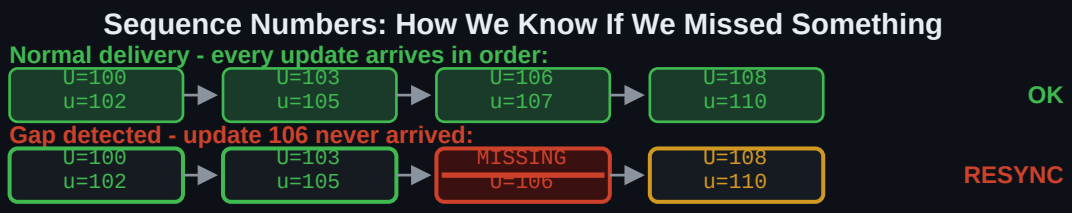

Figure 11 - When a gap appears the book is unrecoverable. We discard it and rebuild from a fresh snapshot.

The tests and what they prove

We wrote four tests that run automatically every time we compile. The first checks that the sequence IDs come through correctly. The second checks that a price and quantity parse to the right values. The third checks that a zero quantity is correctly identified as a deletion signal. The fourth checks that garbage input, a price field containing the word 'bad', produces an error rather than silently using a wrong value. These four tests cover every path through the parsing logic.

Decision	We Chose	Alternative	Why This One
Wire format visibility	Private to binance.rs	Shared across modules	If Binance changes their field names, only one file needs updating. N
Price type	rust_decimal::Decimal	f64 float	Floats store 29500.00 as 29499.999999999996... in binary. That erro
Deletion signal	quantity == 0 check	Separate delete message type	Binance uses zero quantity as their deletion signal. We match that co
Parsing location	All in parse_levels()	Parse at point of use	If a string fails to parse as a number, that is a data error and belongs

Part 7 - orderbook/book.rs: The In-Memory Order Book

This file is the heart of the entire system. Everything else exists to feed data into this structure or to read from it. It holds the current state of the market: every price level where someone is willing to buy or sell, and exactly how much they are willing to trade at that price. It gets updated thousands of times per second and read just as often.

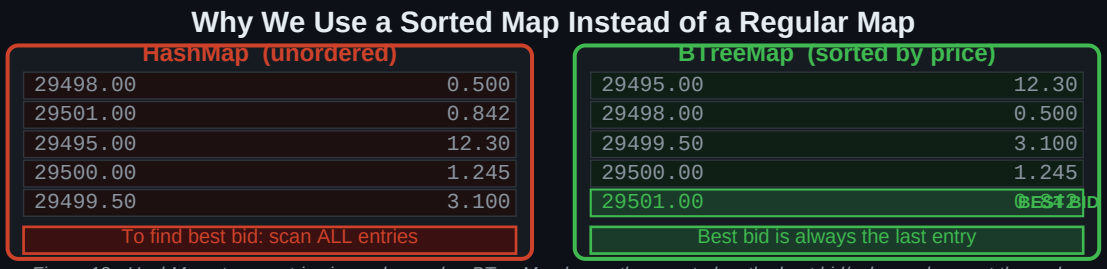

Figure 12 - HashMap stores entries in random order. BTreeMap keeps them sorted so the best bid/ask are always at the ends.

Why the data structure choice is the most important decision in this file

We need to do two things constantly: update a specific price level by its price, and find the best bid and best ask instantly. These two requirements pull in different directions. The structure we chose, called a BTreeMap, is a sorted dictionary. You give it a price as the key and a quantity as the value. It keeps all entries sorted by price at all times, automatically, with every insert and remove. That sorting is what makes finding the best bid and ask free, because they are always at the two ends of the sorted structure.

What it looks like in memory

The bids side of the book is one BTreeMap where each key is a price and each value is how many Bitcoin are available at that price. The map is sorted lowest to highest, so the best bid, the highest buyer, is always sitting at the very back of the map. The asks side is identical in structure. Because asks are also sorted lowest to highest, the best ask, the cheapest seller, is always at the very front.

Figure 13 - Both maps sorted ascending. Best bid is at the back of bids. Best ask is at the front of asks.

What happens when an update arrives

Every update from Binance contains a list of changed price levels for bids and a list for asks. For each one we do exactly one of two things. If the quantity sent is greater than zero, we insert or overwrite that price in the map. If the quantity is zero, we remove that price from the map entirely. The BTreeMap handles both cases with the same operation: insert adds a new entry or replaces an existing one, remove deletes an entry if it exists and does nothing if it does not. We never need to check whether an entry exists first.

Figure 14 - Every incoming price level takes exactly one of two paths. No other cases exist.

Finding the best bid and best ask

When an API request comes in asking for the current best bid, we call one operation on the bids map: give me the last entry. Because the map is sorted ascending by price, the last entry is always the highest price. That is the best bid. Finding the best ask is the mirror image: give me the first entry in the asks map, which is always the lowest price. Neither operation scans the entire map. The time it takes does not grow as the book gets larger.

The spread

The spread is the difference between the best ask and the best bid. It represents the cost of immediately buying and immediately selling: if you bought at the best ask and sold at the best bid right after, you would lose exactly the spread. A tight spread means a healthy, liquid market. A wide spread means the market is thin or uncertain. We calculate it by subtracting the two prices and return None if either side of the book is empty, because there is no spread without both sides.

Figure 15 - The spread is the gap between the highest buyer and lowest seller. Tighter = more liquid market.

The clear method and when it gets used

The clear method wipes both maps completely and resets the sequence counter to zero. This is called in two situations. First, when the WebSocket reconnects after a drop, because the stream of updates we receive is a new stream and we cannot mix it with data from the old stream. Second, when we detect a gap in the sequence numbers, meaning we missed at least one update and our book is now incorrect. In both cases the only safe response is to start completely fresh by fetching a new snapshot from Binance's REST API and rebuilding.

Why the book does not validate sequence numbers itself

The book's only job is to store and update price levels correctly. It does not decide whether an update is valid or whether a gap occurred. That decision lives in the manager, which we write next. This split keeps the book simple, testable, and free of state machine logic. You can test every method on the book without needing a WebSocket connection or caring about Binance's sequencing protocol.

Decision	We Chose	Alternative	Why This One
Price level storage	BTreeMap<Decimal, Decimal>	Vec<(Decimal, Decimal)>	A Vec would need to stay sorted on every insert, which is slow. BTree
Finding best bid/ask	next_back() / next() on BTreeMap	Track separately in variables	Tracking best bid/ask in separate variables means you have to updat
Map key type	Decimal	String or f64	f64 keys would cause rounding collisions where two prices that look t
Sequence validation	Not in this file, done by manager	Inside apply()	Mixing validation logic into the book would make it impossible to test